C++ Chapter 3: Decisions

Contents

C++ Chapter 3: Decisions 1
Read: Think C++ 1
Do: Online Tutorials 1
Casting 1
Tutorial 3.1: Casting 2
If Selection Statement 4
If Else 4
If Else If 5
Tutorial 3.2: Room Area 5
Tutorial 3.3: Ticket to Prize 7
Compound Boolean Expressions 9
Tutorial 3.4: Compound Conditional 9
Tutorial 3.5: Input Validation 11
Assignment Submission 12

Time required: 90 minutes

Read: Think C++

- Chapter 4 Conditionals and Recursion

Do: Online Tutorials

- C++ Conditions
- C++ if, if…else and Nested if…else

Casting

Casting is a conversion process wherein data can be changed from one type to another. C++ has two types of conversions:

Implicit conversion: Conversions are performed automatically by the compiler without the programmer's intervention.

```
int i_variable{10};
// Assigning an int to a float will trigger a conversion to float
float f_variable = i_variable;

// Multiplying or dividing with an int and a double will trigger a
conversion
int num1{3};
double num2{2};
double answer;

// This will be double math as one operand is a double
answer = num1 / num2;
```

Explicit conversion: Conversions are performed only when explicitly specified by the programmer.

```
int num1{3};
in num2{2};
double answer;
// This will be double math as one operand is cast to a double
answer = static_cast<double>num1 / num2;
```

Tutorial 3.1: Casting

Casting is changing or converting a variable value from one data type to another. The following example shows how to use the C++ **static_cast** method.

```cpp
/**
 * @file    casting.cpp
 * @author  William A Loring
 * @version V1.0.0
 * @date    10/17/2022
 * @brief   Use static_cast to cast data type from int to double
 */

#include <iostream>
int main()
{
    int num1{2};
    int num2{45};
    int num3{800};
    double average;

    // If all the operands are integers
    // the result is integer math
    average = (num1 + num2 + num3) / 3;
    std::cout << "Using integer math" << std::endl;
    std::cout << "The average of " << num1 << ", "
              << num2 << " and " << num3 << " is "
              << average << std::endl;

    // If one number in an expression is a double,
    // the result is floating point math
    // static_cast<datatype>(variable)
    average = (static_cast<double>(num1) + num2 + num3) / 3;
    std::cout << "Using double math" << std::endl;
    std::cout << "The average of " << num1 << ", "
              << num2 << " and " << num3 << " is "
              << average << std::endl;
    return 0;
}
```

Example run:

```
Using integer math
The average of 2, 45 and 800 is 282
Using double math
The average of 2, 45 and 800 is 282.333
```

If Selection Statement

```
#include <iostream>
int main()
{
    int age = 19;
    if (age >= 16)
    {
        std::cout << "You can drive!" << std::endl;
    }
    return 0;
}
```

If Else

```
#include <iostream>
int main()
{
    int age = 19;
    if (age >= 16)
    {
        std::cout << "You can drive!" << std::endl;
    }
    else
    {
        std::cout << "You cannot drive yet!" << std::endl;
    }
    return 0;
}
```

If Else If

```
#include <iostream>
int main()
{
    int age;
    std::cout << "Welcome to the Sunset Bar and Grille." << std::endl;
    std::cout << "Please enter your age: ";
    std::cin >> age;

    if (age >= 21)
    {
        std::cout << "Here, have a beer." << std::endl;
    }
    else if (age >= 16)
    {
        std::cout << "Here have a Coke!" << std::endl;
        std::cout << "At least you can drive!" << std::endl;
    }
    else
    {
        std::cout << "Here have a Coke!" << std::endl;
    }
    std::cout << "Thanks for coming to the Sunset Bar and Grille!" <<
std::endl;
    return 0;
}
```

Tutorial 3.2: Room Area

Create, compile, run and attach the following program.

```cpp
/**
 * @file    room_area.cpp
 * @author
 * @version V1.0.0
 * @date    09/12/2021
 * @brief   Calculate square feet of a room
 */
#include <iostream>

int main()
{
    // TODO: Declarations
    // Declare and initialize constants and variables
    const int BIG_ROOM{1000};
    int room_width{0};
    int room_length{0};
    int square_feet{0};

    // TODO: Input
    // Get width and length of the room from the user
    std::cout << "Enter the width of the room in feet: ";
    std::cin >> room_width;
    std::cout << "Enter the length of the room in feet: ";
    std::cin >> room_length;

    // TODO: Calculate
    // Multiply roomWidth by roomLength to get square feet
    square_feet = room_width * room_length;

    // TODO: Output
    // Display results to user
    if (square_feet > BIG_ROOM)
    {
        std::cout << square_feet
                  << " square feet is a big room." << std::endl;
    }
    else
    {
        std::cout << square_feet
                  << " square feet is a normal size room." << std::endl;
    }
    return 0;
}
```

Example run:

```
Enter the width of the room in feet: 20
Enter the length of the room in feet: 45
900 square feet is a normal size room.
```

```
Enter the width of the room in feet: 40
Enter the length of the room in feet: 40
1600 square feet is a big room.
```

Tutorial 3.3: Ticket to Prize

Create, compile, run and attach the following program.

```cpp
/**
 * @file    TicketToPrize.cpp
 * @author
 * @version V1.0.0
 * @date    09/12/2021
 * @brief   Calculate prize based on number of tickets
 */
#include <iostream>

int main()
{
    // Constants for prize levels
    const int FIVE_TICKETS{5};
    const int TEN_TICKETS{10};
    const int FIFTY_TICKETS{50};
    // Store the number of tickets the user has
    int tickets;

    // Prompt the user and get the number of tickets
    std::cout << "How many tickets do you wish to purchase [5, 10, 50]: ";
    std::cin >> tickets;

    // Determine and display the prize based on the number of tickets
    if (tickets == FIVE_TICKETS)
    {
        std::cout << "Not enough tickets - keep trying!";
    }
    // Second condition
    else if (tickets == TEN_TICKETS)
    {
        std::cout << "You win a slinky! Congratulations!";
    }
    // Third condition
    else if (tickets == FIFTY_TICKETS)
    {
        std::cout << "You win a vacuum cleaner! Congratulations!";
    }
    // The user chose a different number than they were prompted for
    else
    {
        std::cout << "Apparently you can't follow directions, you lose.";
    }
    return 0;
}
```

Example run:

```
How many tickets do you wish to purchase [5, 10, 50]:
Not enough tickets - keep trying!
```

```
How many tickets do you wish to purchase [5, 10, 50]: 10
You win a slinky! Congratulations!
```

```
How many tickets do you wish to purchase [5, 10, 50]: 50
You win a vacuum cleaner! Congratulations!
```

```
How many tickets do you wish to purchase [5, 10, 50]: 2
Apparently you can't follow directions, you lose.
```

Compound Boolean Expressions

Simple Boolean expressions, each involving one relational operator, can be combined into more complex Boolean expressions using the logical operators **&&** (and), **||** (or), and **!** (not). A combination of two or more Boolean expressions using logical operators is called a compound Boolean expression.

&& (and)

```
\\ && (and operator) Both sides must be true
sun_is_shining = true
no_rain = true
if sun_is_shining && no_rain
    go outside
```

|| (or)

```
\\ || (or operator) Only one side must be true
sun_is_shining = true
no_rain = false
if sun_is_shining || no_rain
    go outside
```

Tutorial 3.4: Compound Conditional

The following program demonstrates compound conditionals.

```cpp
/**
 * @file    compound_conditional.cpp
 * @author
 * @version V1.0.0
 * @date    10/17/2022
 * @brief   More than one condition in a decision statement
 */

#include <iostream>
int main()
{
    int age;
    std::cout << "Enter your age: ";
    std::cin >> age;

    // && and - both conditions must be true
    if (age >= 12 && age <= 50)
    {
        std::cout << "Young" << std::endl;
    }
    else
    {
        std::cout << "Not Young" << std::endl;
    }

    // || or - either condition must be true
    if (age < 12 || age > 50)
    {
        std::cout << "Eligible for the offer" << std::endl;
    }
    else
    {
        std::cout << "Not eligible for the offer" << std::endl;
    }
    return 0;
}
```

```
Enter your age: 21
Young
Not eligible for the offer
```

```
Enter your age: 51
Not Young
Eligible for the offer
```

Tutorial 3.5: Input Validation

Using input validation is defensive programming.

Defensive programming is a software development approach that aims to anticipate and mitigate potential issues, errors, and vulnerabilities in code. It involves strategies and practices to enhance the robustness and reliability of software systems. Some key aspects include input validation, error handling, and thorough testing to ensure that software can withstand unexpected situations and minimize the risk of failures.

The following example shows how to prevent a divide by 0 error.

```cpp
#include <iostream>

int main()
{
    int a, b, c;
    std::cout << "Enter a number: " << std::endl;
    std::cin >> a;

    std::cout << "Enter a second number: " << std::endl;
    std::cin >> b;
    if (b == 0)
    {
        std::cout
            << "Invalid denominator, you cannot divide by 0"
            << std::endl;
    }
    else
    {
        c = a / b;
        std::cout << c << std::endl;
    }
    return 0;
}
```

Example runs:

```
Enter a number: 4
Enter a second number: 0
Invalid denominator, you cannot divide by 0
```

```
Enter a number: 5
Enter a second number: 4
1
```

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.